Thông tin chung

INSTRUCTIONS

Read the instructions below carefully before answering the questions:

- You are allowed to use the materials on your computer, notebooks, and textbooks.
- You are **NOT allowed** to use electronic devices or any other means to share data with others.
- Do not arbitrarily disconnect the **FPTU_EXAMONLINE** network while taking the test. If any network problems occur, immediately notify the exam supervisor for assistance.

Besides the conditions above, students must fulfill the following requirements:

1. Use **Visual Studio Code** (VS Code), **MongoDB Compass**, **Postman** or **Thunder Client** (an extension for VSCode) tools (offline) to complete the exam
2. Use the **NodeJS platform version 22.11.0**
3. Use the available browser software on your computer

This part is very important, please read it carefully and follow the instructions

- You are provided with the data files located in the **given** directory.
- You must use the pre-provided solution sent via email and install the additional library included in the accompanying request.
- You are **not allowed** to download any additional libraries while taking the test
- Violating any of the above rules makes your exam invalid

Instructions for submission

- Before submitting the solution, delete the [**node_modules**] folders in your test to reduce the space and meet the allowed capacity of the **PEA_Client** software
- Use the **PEA_Client** software to browse the **resource** directory to complete the submission process.

System Requirements

Use **MongoDB Compass** to create a database named **"SDN_Trial"**. Create **collections** matching the data in the **given** directory, then import the data into the collections you created.

Collections (from the **given** directory):

```
movies
rooms
showtimes
tickets
users
```

Using the given, build a RESTful API application that fulfills the requirements of the problem:

Note: 0 will be given to the work that

- Not using connection string to MongoDB in the **.env** file
- Not configuring the root path of RESTful API Web App at: http://localhost:9999

All endpoints must:

- Validate input data
- Apply business rules
- Return appropriate HTTP status codes
- Return properly structured JSON responses

Protected routes must implement authorization middleware based on the **access_key** from request headers.

Context

The system is an online cinema ticket booking platform whose data is organized into five collections: **movies**, **rooms**, **showtimes**, **tickets** and **users**.

Students are required to build the API endpoints using **ExpressJS, MongoDB, and Mongoose**. Each task specifies an API operation that must be implemented, including input validation, business rules, database processing, and proper JSON response formatting. Students must use **Postman (offline), or Thunder Client** to test their APIs.

Question 1 (1.5 points): Retrieve All Rooms (Sorted)

Endpoint:

— Tài liệu được thành viên đăng tải tại ***FUOVERFLOW*** và được biên soạn lại để hỗ trợ ôn tập —

```
GET /api/rooms
```

Business Rules:

- Sort rooms by **roomName** in descending order
- Return:
  - id
  - name
  - totalSeats
  - vipCount (number of VIP seats)

Expected Response:

- 200 (OK)

```
[
    {
        "id": "65fb0000000000000000005",
        "name": "Room E",
        "totalSeats": 6,
        "vipCount": 3
    },
    ......
]
```

Question 2 (2.5 points): Retrieve Movie Details with Showtimes

Objective:

Fetch detailed information about a specific movie, including all of its showtimes.

Endpoint:

```
GET /api/movies/:movieId
```

Business Rules:

- Validate movieId format using ObjectId validation.
- Return 400 if movieId is invalid.
- Return 404 if movie does not exist.
- Retrieve all showtimes associated with the movie.
- Populate room information to extract room name.

Expected Response:

- 400 (Bad Request):

```
{
    "error": "Invalid movieId: 65fa0000000000000000000"
}
```

- 404 (Not Found):

```
{
    "error": "Movie with id: 65fa00000000000000000009 not found"
}
```

- 200 (OK): Test with **movieId=" 65fa00000000000000000001"**

```
{
    "title": "Avengers",
    "duration": 180,
    "description": "Superheroes assemble to save the world.",
    "showtimes": [
        {
            "roomName": "Room A",
            "startTime": "2026-03-10T18:30:00.000Z",
            "price": 90000
        },
        {
            "roomName": "Room B",
            "startTime": "2026-03-11T18:00:00.000Z",
            "price": 85000
        },
        {
            "roomName": "Room A",
            "startTime": "2026-03-10T20:00:00.000Z",
            "price": 90000
```

```
    ]
}
```

Question 3 (3.5 points): Create Showtime (Admin Only)

Endpoint:

```
POST /api/showtimes
```

Authorization: 0 points if no authorization

- Header:
  - Example: **access_key: 123456abcdef**
- Rules:
  - **access_key** must exist
  - Role must be "admin"

Request Body:

```
{
    "movieId": "65fa0000000000000000001",
    "roomId": "65fb0000000000000000001",
    "startTime": "2026-03-10T19:00:00.000Z",
    "price": 95000
}
```

Business Rules:

- Movie must exist and **status = NOW_SHOWING**
- Room must exist
- Calculate **endTime = startTime + movie.duration**
- Reject if showtime overlaps in same room. Overlap condition:
  - **existing.startTime < newEndTime AND existing.startTime < newEndTime**

Expected Response:

- 403 (Forbidden): No access_key

```
{
    "error": "No access key"
}
```

- 403 (Forbidden): Invalid access_key

```
{
    "error": "Invalid or unauthorized access key"
}
```

- 403 (Forbidden): Invalid admin role

```
{
    "error": "Admin access required"
}
```

- 400 (Bad Request): Fields required

```
{
    "error": "movieId, roomId, startTime, price required"
}
```

- 409 (Conflict): Showtime overlap

```
{
    "error": "Showtime overlaps with existing"
}
```

- 201 (Created):

```
{
    "_id": "699b2384403fbfae0baf24a4",
    "movieId": "65fa0000000000000000001",
    "roomId": "65fb0000000000000000001",
    "startTime": "2026-03-10T19:00:00.000Z",
```

```
    "endTime": "2026-03-10T22:00:00.000Z",
    "price": 95000
}
```

Question 4 (2.5 points): Revenue Report (Admin Only)

— Tài liệu được thành viên đăng tải tại **FUOVERFLOW** và được biên soạn lại để hỗ trợ ôn tập —

Endpoint:

```
GET /api/reports/revenue
```

Authorization:

- Requires valid **access_key**
- Students must: Read **access_key** from request headers
  - Example: access_key: 123456abcdef

Authorization Rules:

- Validate access_key
- Find user by access_key
- Reject if not found → 403
- Role must be "admin"
- Otherwise → 403

Business Rules:

- Only aggregate tickets with status = "paid"
- Group results by movie
- Calculate:
  - ticketCount
  - total revenue

Expected Response:

- 403 (Forbidden): No access_key

```
{
    "error": "No access key"
}
```

- 403 (Forbidden): Invalid access_key

```
{
    "error": "Invalid or unauthorized access key"
}
```

- 403 (Forbidden): Invalid admin role

```
{
    "error": "Admin access required"
}
```

- 200 (OK):

```
[
    {
        "movieTitle": "Avengers",
        "ticketCount": 3,
        "revenue": 270000
    },
    {
        "movieTitle": "Avatar",
        "ticketCount": 1,
        "revenue": 100000
    },
    ......
]
```

---- END ----